קטע קוד A

```
class Expression {
public:
    Expression() {/* print(); */}
    virtual ~Expression() {/* cout << *this << endl; */}
    virtual double calc()const = 0;
    virtual void print(ostream& out = cout)const = 0;
    friend ostream& operator<<(ostream& out, const Expression& e)
       { e.print(out); return out; }
};

class Number final : public Expression {
    double _d;
public:
    Number(double d) : _d(d) {}
    ~Number() { /* cout << _d << endl; */ }
    virtual double calc()const override {return _d;}
    virtual void print(ostream& out = cout)const override {
        out << _d;
    }
};

class BinaryExpression: public Expression {
    char _sign;
    Expression *_e1, *_e2;
public:
    BinaryExpression(char sign, Expression* e1, Expression* e2):
      _sign(sign), _e1(e1), _e2(e2) {/*cout << *this << endl;*/}
    ~BinaryExpression() {
        delete _e1; delete _e2;
    }
    BinaryExpression(const BinaryExpression&) = delete;
    BinaryExpression& operator=(const BinaryExpression&) = delete;
    virtual double calcImpl(double d1, double d2)const = 0;
    virtual double calc()const override final {
        return calcImpl(_e1->calc(), _e2->calc());
    }
    virtual void print(ostream& out = cout)const override final {
        out << '(' << *_e1 << ' ' << _sign << ' ' << *_e2 << ')';
    }
};

class Sum: public BinaryExpression  {
public:
    Sum(Expression* e1, Expression* e2):
        BinaryExpression('+', e1, e2) {}
    virtual double calcImpl(double d1, double d2)const override
        { return d1 + d2; }
};
```

להלן מספר main אפשריים:

A1 main

```
int main() {
  Expression* e = new Sum(
          new Number(3.5), new Sum(new Number(1), new Number(3))
      );
  cout << *e << " = " << e->calc() << endl;
  delete e;
}
```

A2 main

```
int main() {
  Expression* e1 = new Number(3.5);
  Number n2 = *(static_cast<Number*>(e1));
  cout << *e1 << " = " << e1->calc() << endl;
  cout << n2 << " = " << n2.calc() << endl;
  delete e1;
}
```

A3 main

```
int main() {
  Expression* e1 = new Sum(new Number(1), new Number(2));
  Sum s2 = *(dynamic_cast<Sum*>(e1));
  cout << *e1 << " = " << e1->calc() << endl;
  cout << s2 << " = " << s2.calc() << endl;
  delete e1;
}
```

A4 main

```
int main() {
  list<Expression*> expressions =
     {new Number(2.5), new Sum(new Number(1), new Number(2))};
  Expression* max = nullptr;
  double maxVal;
  for_each(expressions.begin(), expressions.end(),
      [&max, &maxVal](Expression* e){
          double currVal = e->calc();
          if(max == nullptr || currVal > maxVal) {
             delete max;
             max = e;
             maxVal = currVal;
          }
          else {
             delete e;
          }
     });
  cout << *max << " = " << maxVal;
  delete max;
}
```

קטע קוד B

```
 1. struct Expression {
 2.   virtual ~Expression() {}
 3.   virtual void print(ostream& out = cout)const = 0;
 4.   virtual operator double()const = 0; // casting to double
 5.   friend ostream& operator<<(ostream& out, const Expression& e)
 6.     { e.print(out); return out; }
 7. };
 8.
 9. class Number final : public Expression {
10.   double _d;
11. public:
12.   Number(double d) : _d(d) {}
13.   virtual operator double()const override {return _d;}
14.   virtual void print(ostream& out = cout)const { out << _d; }
15. };
16.
17. class BinaryExpression: public Expression {
18.   char _sign;
19.   Expression *_e1 = nullptr, *_e2 = nullptr;
20. public:
21.   BinaryExpression(char sign, Expression* e1, Expression* e2):
22.       _sign(sign), _e1(e1), _e2(e2) {}
23.   ~BinaryExpression() { delete _e1; delete _e2; }
24.   BinaryExpression(BinaryExpression&& other) {
25.      std::swap(_e1, other._e1); std::swap(_e2, other._e2);
26.      _sign = other._sign;
27.   }
28.   virtual double calc(double d1, double d2)const = 0;
29.   virtual operator double()const override final {
30.      return calc(*_e1, *_e2);
31.   }
32.   virtual void print(ostream& out = cout)const override final {
33.      out << '(' << *_e1 << ' ' << _sign << ' ' << *_e2 << ')';
34.   }
35. };
36.
37. struct Sum: public BinaryExpression  {
38.   template<class E1, class E2>
39.   Sum(E1&& e1, E2&& e2): BinaryExpression('+',
40.         new E1(std::move(e1)), new E2(std::move(e2))) {}
41.   virtual double calc(double d1, double d2)const override
42.         { return d1 + d2; }
43. };
44.
45. void print(const Expression& e) {
46.     cout << e << " = " << (double)e << endl;
47. }
48.
49. int main() {
50.     print( Sum(Number(1), Number(2)) );
51.     print( Sum(Sum(Number(1), Number(2)), Number(3)) );
52. }
```

בהצלחה !